

CSE303 (Statistics for Datascience)

Dataset Analysis

Report of Dataset

Submitted By:

Afsin Sultana

2022-1-60-113

Submitted to:

Puja Chakrabarty

Lecturer, East West University

Project title: LoanData Analysis

What are the types of data? (e.g. Qualitative vs Quantitative; Nominal vs Ordinal; Discrete vs Continuous)

Answer:

Qualitative: It is like descriptive and categorical data. It can be non-numeric. For example: gender, loan_status etc

Quantitative: It is numerical value or measurable quantities. For example: loan_amount, income etc.

Nominal: This is like categorical with no order value or ranking

Ordinal: It can be in meaningful order

Discrete: Countable numbers with no fraction

Continuous: Any value with a range.

What is balanced or imbalanced dataset? How does it might affect the performance of a model?

Answer:

Balanced Dataset: All target classes have approximately equal number of samples.

Imbalanced Dataset: One class has much more data than the others.

Imbalanced data is problematic. As it Bias toward Majority Class and Poor Performance on Minority Class.

It effects on performance like accuracy Looks high, but misleading, recall , f1 score drops significantly.

Write a short description of your task and dataset, such as what are the column in your dataset, their type, what are the range or categories of the values in each column etc.

Answer:

My dataset is Loan_data preprocessed . IT has 3 csv file. There are column named age , employ, ed,adress, income , debtinc, creditdebt, otherdebt, default. All of them are numeric data. There is no non-numeric value.

In 1st csv file :

```
Shape of dataset: (700, 9)
Column info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 700 entries, 0 to 699
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   age         680 non-null    float64
 1   employ      700 non-null    int64
 2   address     700 non-null    int64
 3   income      663 non-null    float64
 4   debtinc     700 non-null    float64
 5   creditdebt  700 non-null    float64
 6   othdebt     700 non-null    float64
 7   ed          680 non-null    float64
 8   default     700 non-null    int64
dtypes: float64(6), int64(3)
memory usage: 49.3 KB
None

First 5 rows:
   age  employ  address  income  debtinc  creditdebt  othdebt  ed  default
0  41.0      17      12    176.0      9.3   11.359392   5.008608  3.0        1
1  27.0      10       6     51.0     17.3    1.382202   4.880798  1.0        0
2  40.0      15       7     NaN     5.5    0.856075    2.168925  1.0        0
3  41.0      15      14    120.0      2.9    2.658720    0.821280  NaN        0
4  24.0       2       0     20.0     12.3    1.787436    3.056564  2.0        1

/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1458: RuntimeWarning: invalid value encountered in greater
has_large_values = (abs_vals > 1e5).any()
/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1459: RuntimeWarning: invalid value encountered in less
has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
/usr/local/lib/python3.11/dist-packages/pandas/io/formats/format.py:1459: RuntimeWarning: invalid value encountered in greater
has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
```

df.dtypes

```
age          float64
employ       int64
address      int64
income       float64
debtinc      float64
creditdebt   float64
othdebt      float64
ed           float64
default      int64
dtype: object
```

2nd CSV file:

Shape of dataset: (485, 9)

Column info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 485 entries, 0 to 484

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	age	485 non-null	int64
1	employ	485 non-null	int64
2	address	485 non-null	int64
3	income	485 non-null	float64
4	debtinc	485 non-null	float64
5	creddebt	485 non-null	float64
6	othdebt	485 non-null	float64
7	ed	485 non-null	int64
8	default	485 non-null	int64

dtypes: float64(4), int64(5)

memory usage: 34.2 KB

None

First 5 rows:

	age	employ	address	income	debtinc	creddebt	othdebt	ed	default
0	29	11	7	32.0	6.0	0.927360	0.992640	1	0
1	28	1	3	26.0	12.4	0.377208	2.846792	4	0
2	34	16	3	75.0	10.4	3.954600	3.845400	1	0
3	51	31	14	249.0	7.8	4.272840	15.149160	2	0
4	40	13	11	102.0	18.9	6.226794	13.051206	2	1

```
[6]: 1 df2.dtypes
```

```
[6]: age          int64
     employ      int64
     address     int64
     income      float64
     debtinc     float64
     creddebt    float64
     othdebt     float64
     ed          int64
     default     int64
     dtype: object
```

3rd csv file:

```

Shape of dataset: (700, 9)
Column info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 700 entries, 0 to 699
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0    age        681 non-null   float64
1    ed          680 non-null   float64
2    employ      700 non-null   int64
3    address     700 non-null   int64
4    income      663 non-null   float64
5    debtinc     700 non-null   float64
6    creddebt    700 non-null   float64
7    othdebt     700 non-null   float64
8    default     700 non-null   object
dtypes: float64(6), int64(2), object(1)
memory usage: 49.3+ KB
None

First 5 rows:
   age  ed  employ  address  income  debtinc  creddebt  othdebt  default
0  41.0  3.0     17     12   176.0     9.3  11.359392  5.008608         1
1  27.0  1.0     10      6    31.0    17.3   1.362202  4.000798         0
2  40.0  1.0     15      7     NaN     5.5   0.856075  2.168925         0
3  41.0  NaN     15     14   120.0     2.9   2.658720  0.821280         0
4  24.0  2.0      2      0    28.0    17.3   1.787436  3.056564         1

```

****Types of dataset 3 ****

```

[7]: 1 df3.dtypes

[7]: age          float64
     ed          float64
     employ       int64
     address      int64
     income       float64
     debtinc      float64
     creddebt     float64
     othdebt      float64
     default      object
     dtype: object

```

All of them are balanced.

If any column contains continuous numerical values then calculate the mean, median, variance and standard deviation of that column, otherwise if the values are categorical or discrete then count the frequency and percentage of each type of values.

```
[4]: 1 # Shows both numeric and categorical summaries
      2
      3 print("Columns:", df.columns.tolist())
      4 df.describe(include='all')
      5
```

Columns: ['age', 'employ', 'address', 'income', 'debtinc', 'creddebt', 'othdebt', 'ed', 'default']

```
[4]:
```

	age	employ	address	income	debtinc	creddebt	othdebt	ed	default
count	680.000000	700.000000	700.000000	663.000000	700.000000	700.000000	700.000000	680.000000	700.000000
mean	34.750000	8.388571	8.268571	45.74359	10.260571	1.553553	3.058209	1.717647	0.261429
std	7.973215	6.658039	6.821609	37.44108	6.827234	2.117197	3.287555	0.925652	0.439727
min	20.000000	0.000000	0.000000	14.00000	0.400000	0.011696	0.045584	1.000000	0.000000
25%	28.000000	3.000000	3.000000	24.00000	5.000000	0.369059	1.044178	1.000000	0.000000
50%	34.000000	7.000000	7.000000	34.00000	8.600000	0.854869	1.987567	1.000000	0.000000
75%	40.000000	12.000000	12.000000	54.50000	14.125000	1.901955	3.923065	2.000000	1.000000
max	56.000000	31.000000	34.000000	446.00000	41.300000	20.561310	27.033600	5.000000	1.000000

```
[8]: 1 # Shows both numeric and categorical summaries
      2
      3 print("Columns:", df2.columns.tolist())
      4 df2.describe(include='all')
```

Columns: ['age', 'employ', 'address', 'income', 'debtinc', 'creddebt', 'othdebt', 'ed', 'default']

```
[8]:
```

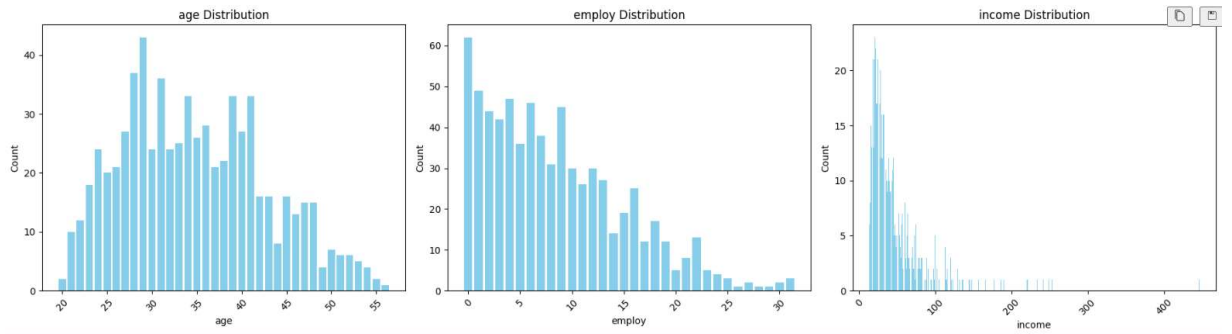
	age	employ	address	income	debtinc	creddebt	othdebt	ed	default
count	485.000000	485.000000	485.000000	485.000000	485.000000	485.000000	485.000000	485.000000	485.000000
mean	35.053608	8.239175	8.501031	44.448802	10.217320	1.407004	2.938476	1.740206	0.249485
std	8.003177	6.557479	7.013159	31.604135	6.747319	1.517178	2.964370	0.948320	0.433161
min	21.000000	0.000000	0.000000	14.000000	0.600000	0.014835	0.045584	1.000000	0.000000
25%	29.000000	3.000000	3.000000	24.000000	5.000000	0.376852	1.054992	1.000000	0.000000
50%	34.000000	7.000000	7.000000	36.000000	8.600000	0.856075	1.947014	1.000000	0.000000
75%	41.000000	12.000000	13.000000	51.000000	14.000000	1.874560	3.817908	2.000000	0.000000
max	56.000000	31.000000	34.000000	249.000000	36.600000	9.600480	18.269130	5.000000	1.000000

```
has_small_values = ((abs_vals < 10 ** (-self.digits)) & (abs_vals > 0)).any()
```

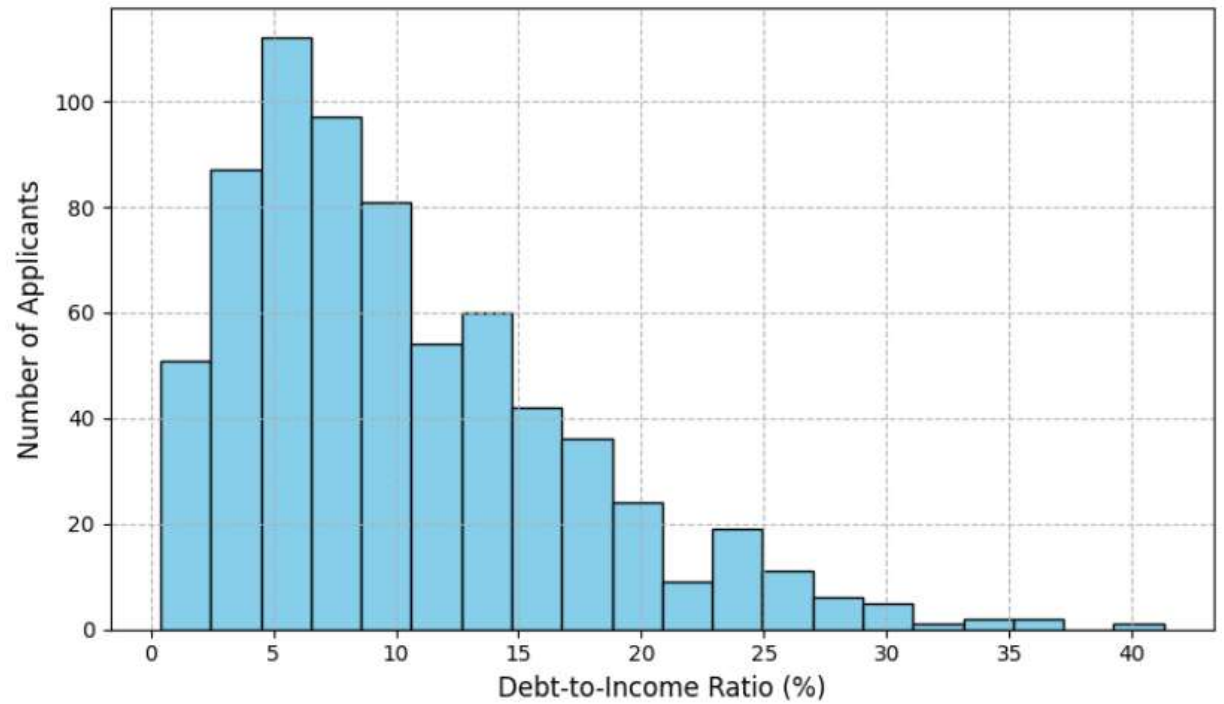
```
[9]:
```

	age	ed	employ	address	income	debtinc	creddebt	othdebt	default
count	681.000000	680.000000	700.000000	700.000000	663.000000	700.000000	700.000000	700.000000	700
unique	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4
top	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0
freq	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	515
mean	34.898678	1.717647	8.388571	8.268571	45.74359	10.260571	1.553553	3.058209	NaN
std	8.861849	0.925652	6.658039	6.821609	37.44108	6.827234	2.117197	3.287555	NaN
min	20.000000	1.000000	0.000000	0.000000	14.00000	0.400000	0.011696	0.045584	NaN
25%	28.000000	1.000000	3.000000	3.000000	24.00000	5.000000	0.369059	1.044178	NaN
50%	34.000000	1.000000	7.000000	7.000000	34.00000	8.600000	0.854869	1.987567	NaN
75%	40.000000	2.000000	12.000000	12.000000	54.50000	14.125000	1.901955	3.923065	NaN
max	136.000000	5.000000	31.000000	34.000000	446.00000	41.300000	20.561310	27.033600	NaN

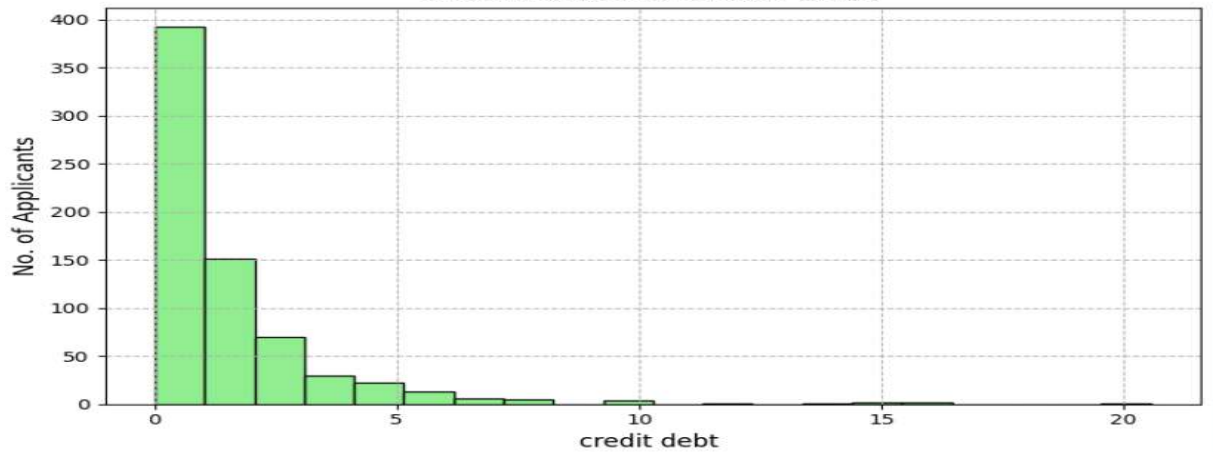
Charts for csv file –1:

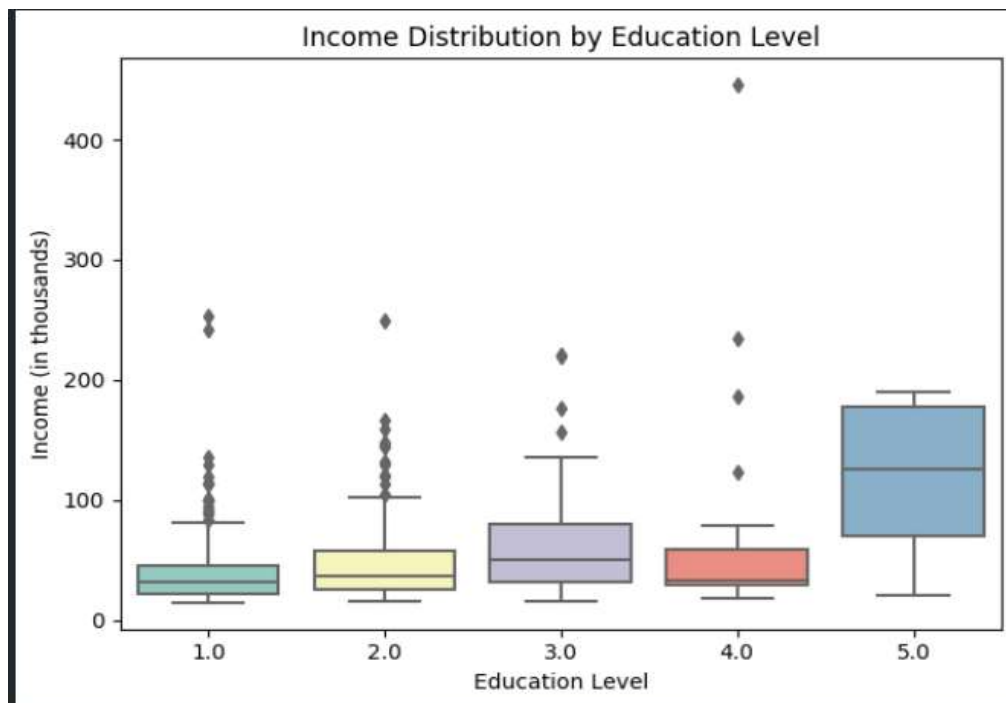
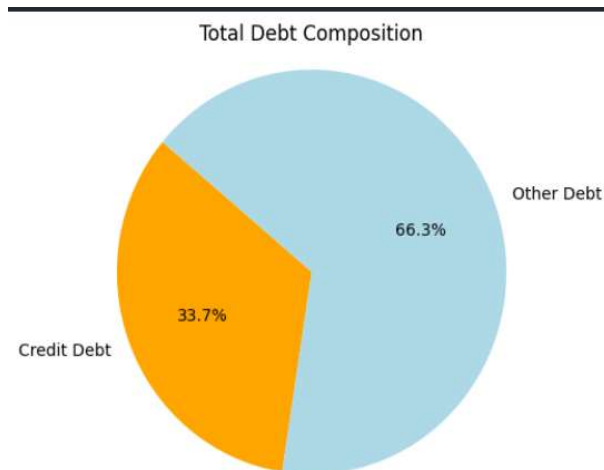


Distribution of Debt-to-Income Ratio

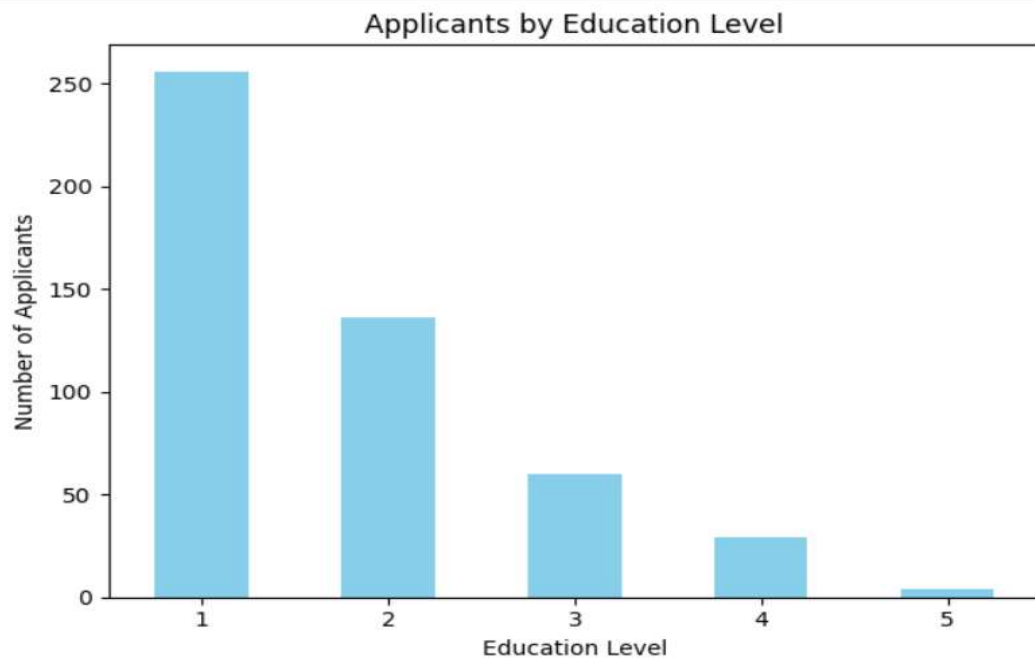
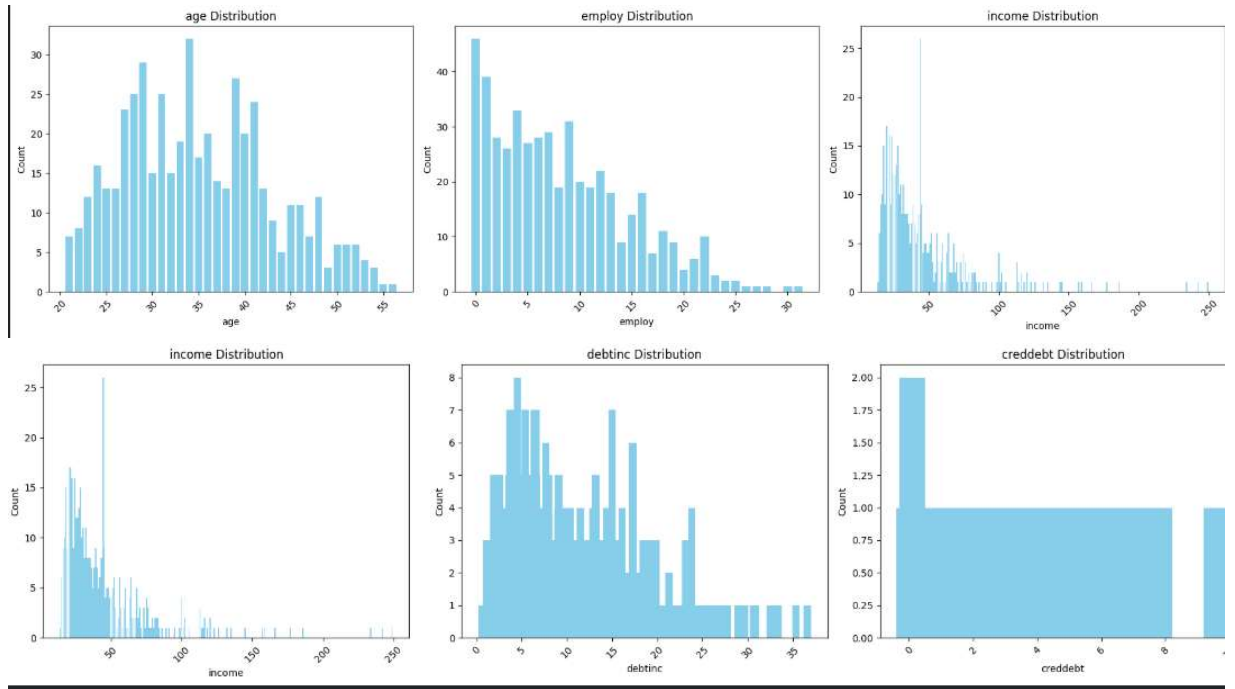


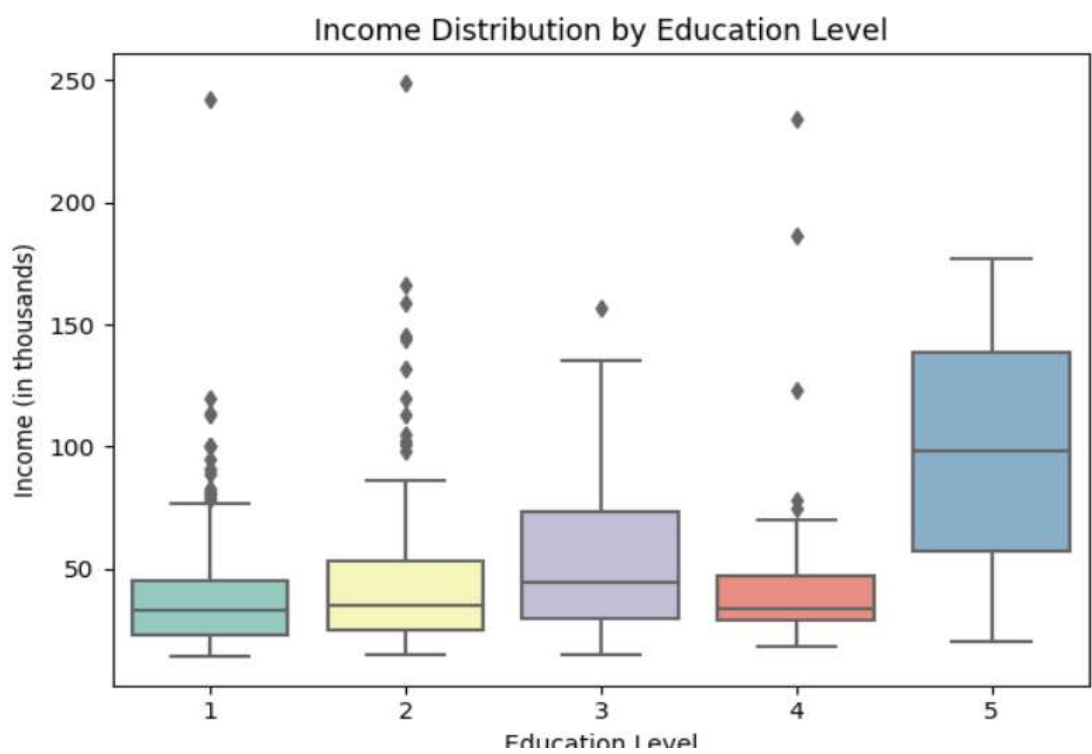
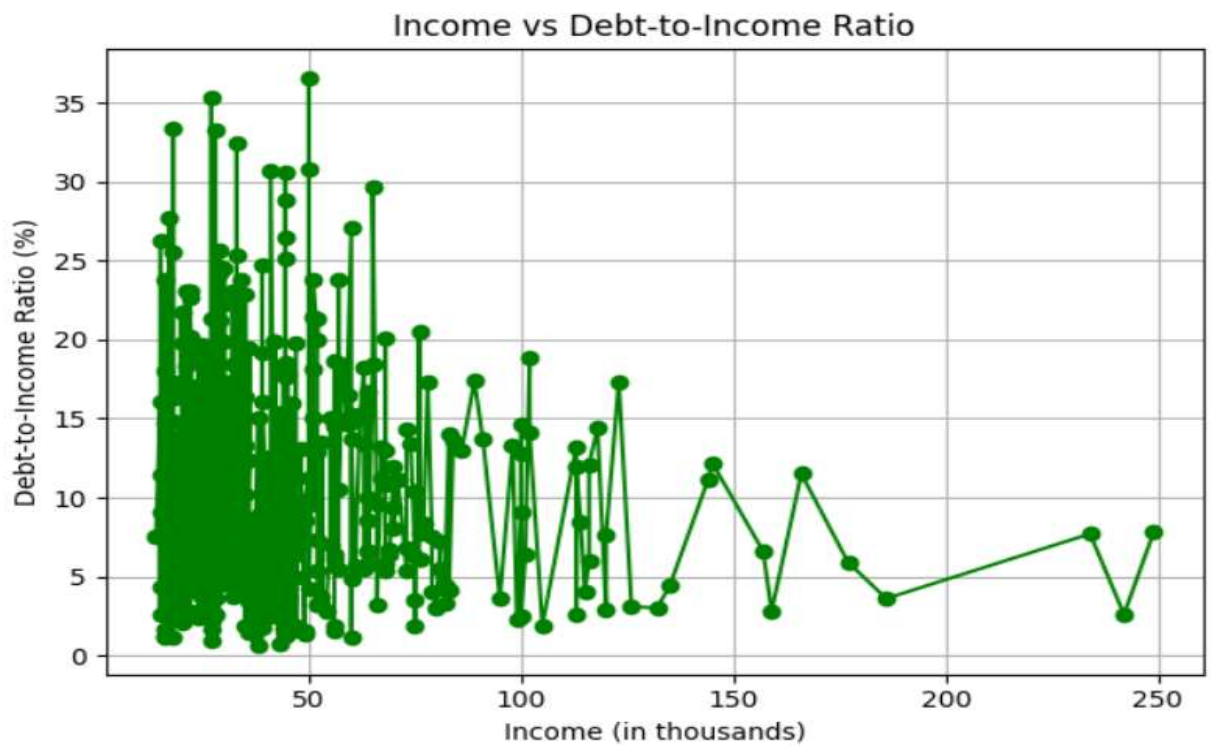
Distribution of Credit Debt



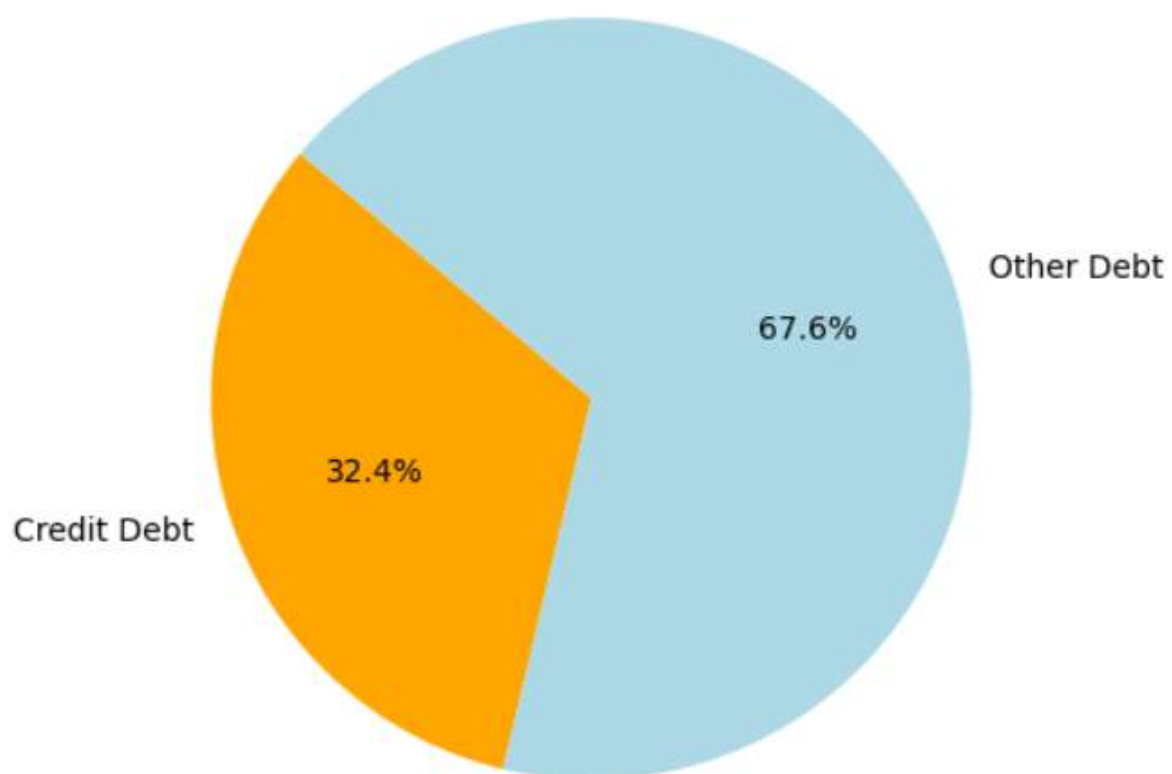


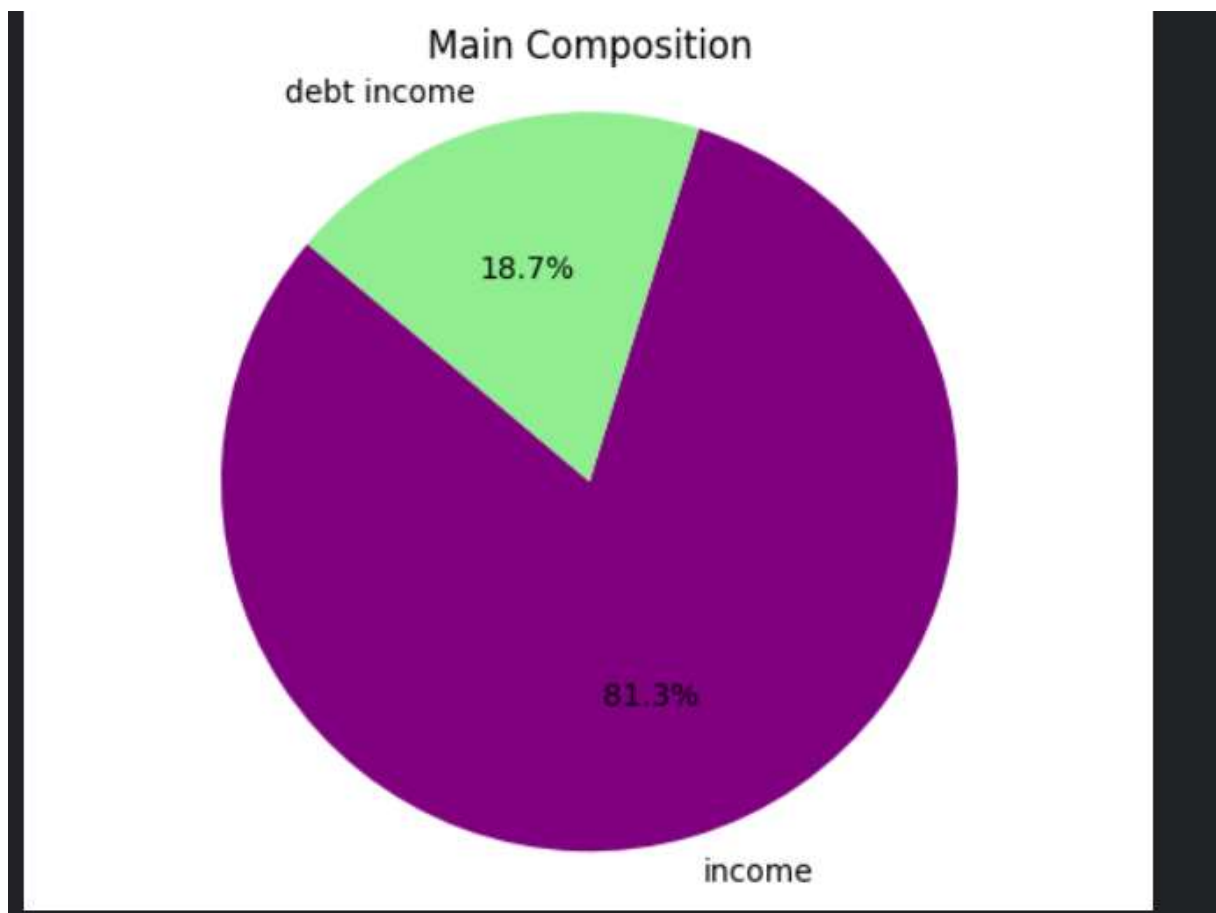
2nd csv file charts:





Total Debt Composition





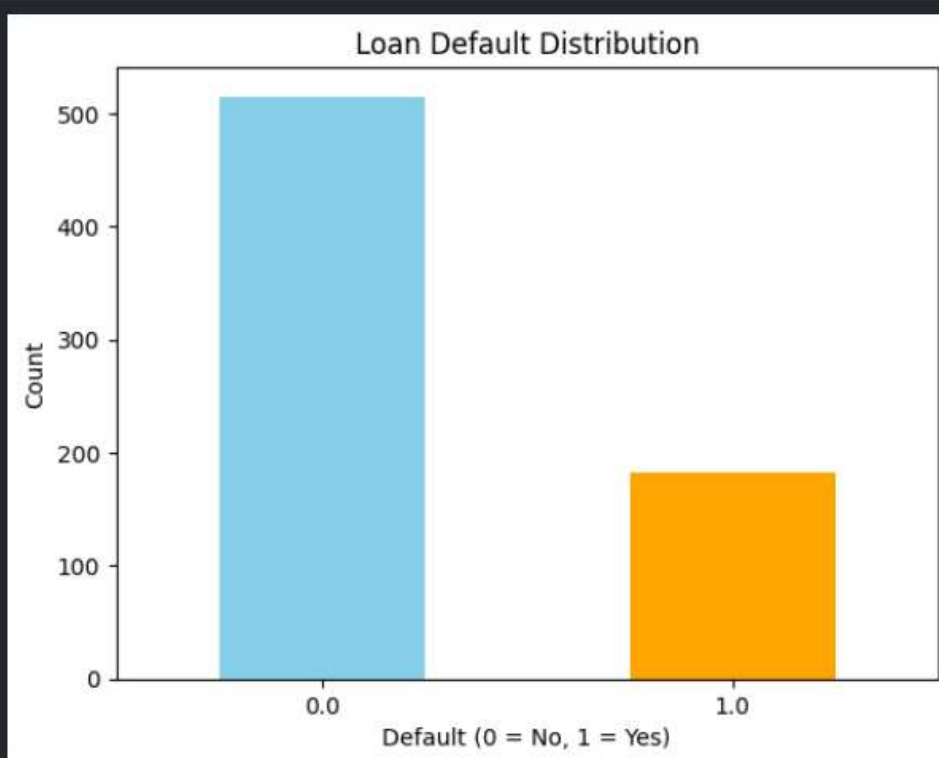
3rd CSV file(charts) :

Bar chart of default frequency (Categorical 0/1)

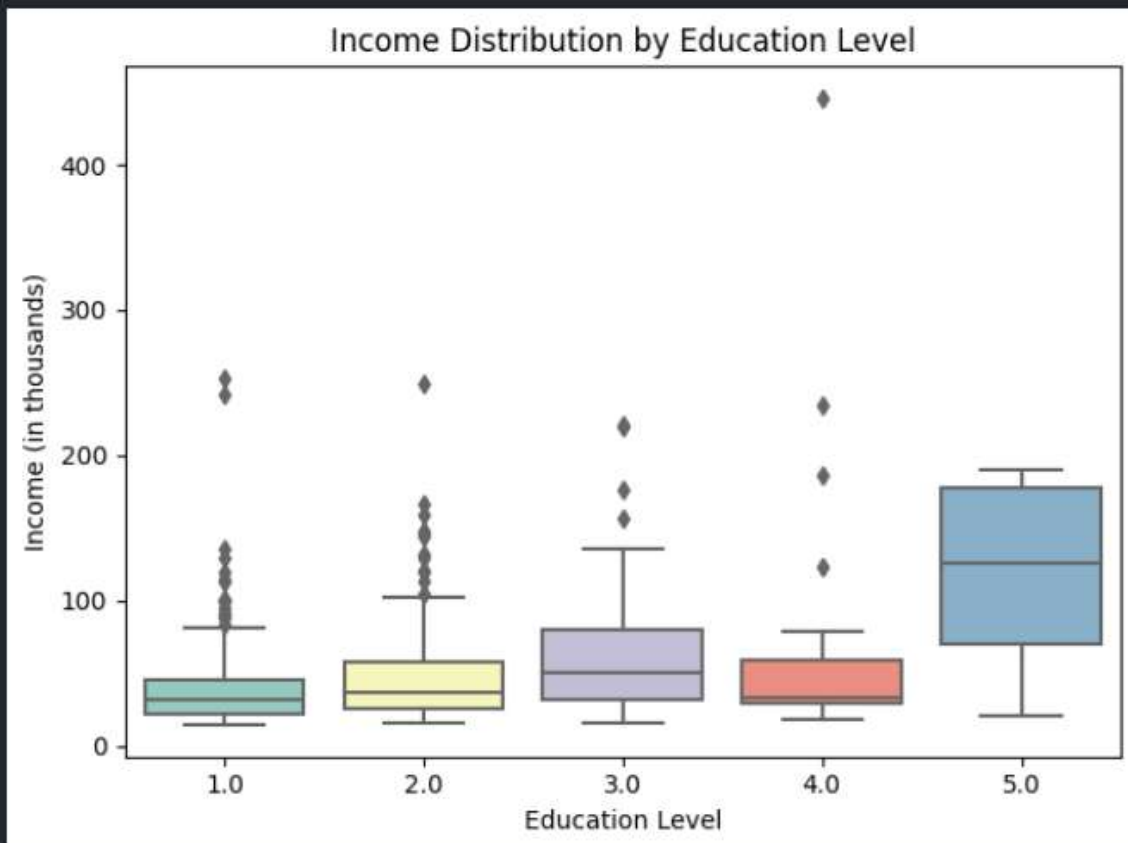
```
df3['default'].value_counts().plot(kind='bar', color=['skyblue', 'orange'])  
plt.title("Loan Default Distribution")  
plt.xlabel("Default (0 = No, 1 = Yes)")  
plt.ylabel("Count")  
plt.xticks(rotation=0)  
plt.show()
```

[42]

...



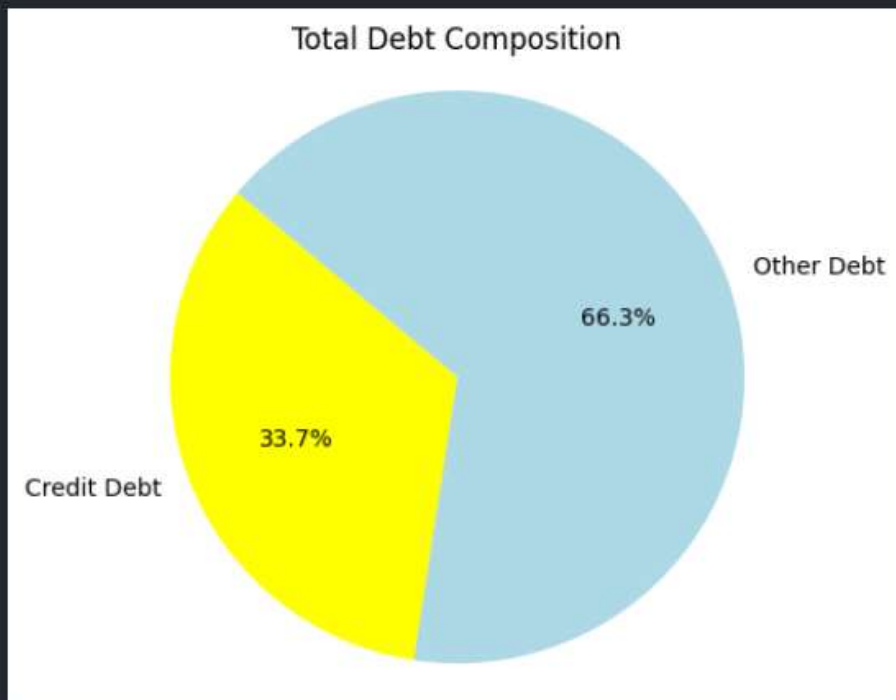
```
# Boxplot: income distribution by education
sns.boxplot(x='ed', y='income', data=df3, palette='Set3')
plt.title('Income Distribution by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Income (in thousands)')
plt.tight_layout()
plt.show()
```



```
total_creddebt44 = df3['creddebt'].sum()
total_othdebt44 = df3['othdebt'].sum()

plt.pie([total_creddebt44, total_othdebt44],
        labels=['Credit Debt', 'Other Debt'],
        autopct='%1.1f%%',
        colors=['yellow', 'lightblue'],
        startangle=140)

plt.title('Total Debt Composition')
plt.axis('equal')
plt.show()
```



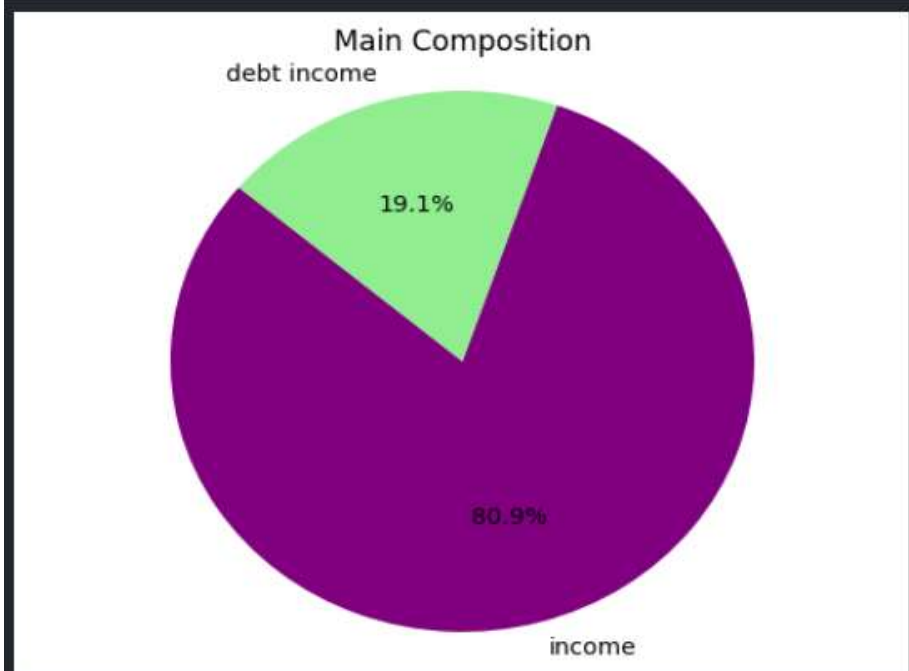
```

t_in = df3['income'].sum()
t_debtin = df3['debtinc'].sum()

plt.pie([t_in ,t_debtin],
        labels=['income', 'debt income'],
        autopct='%1.1f%%',
        colors=['purple', 'lightgreen'],
        startangle=140)

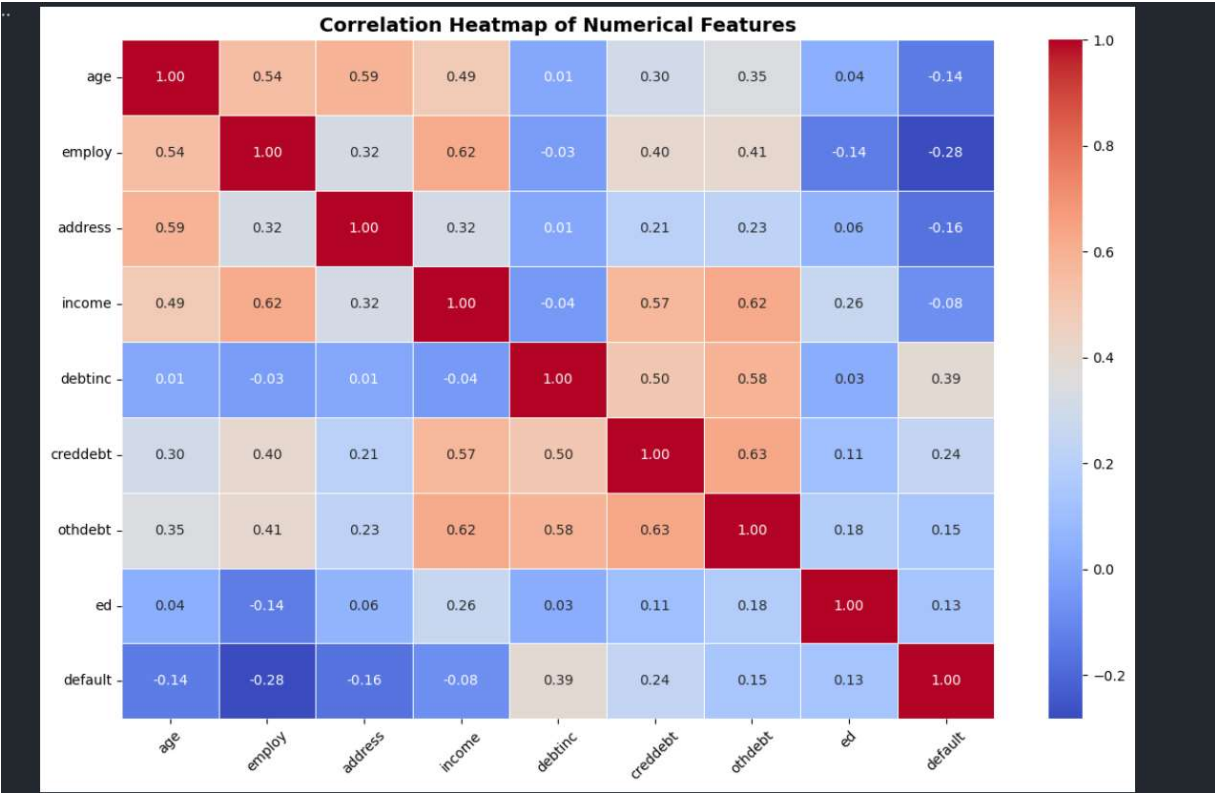
plt.title('Main Composition')
plt.axis('equal')
plt.show()

```



Create a correlation heat-map with each column.

1st csv:



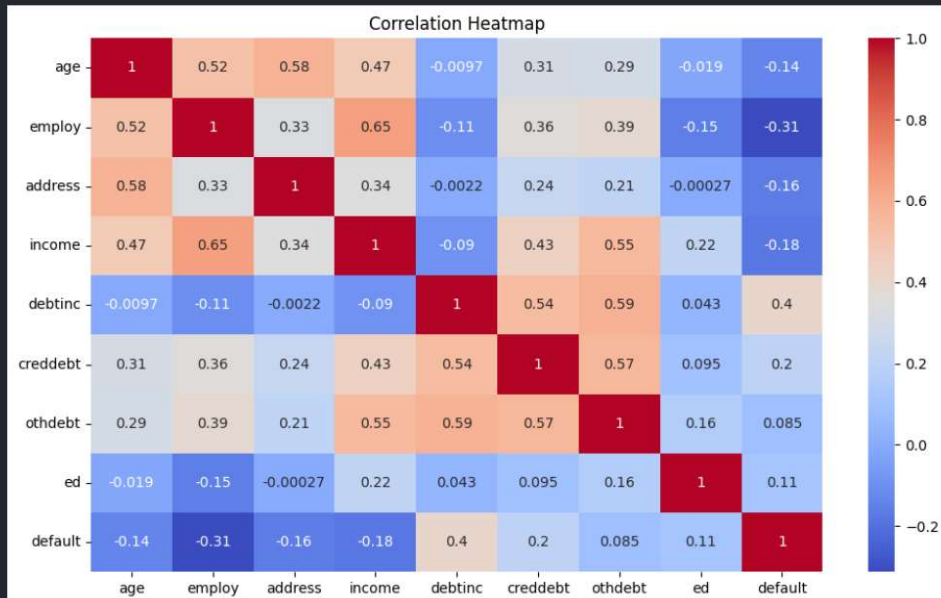
2nd csv file:

```

import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
sns.heatmap(df2.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()

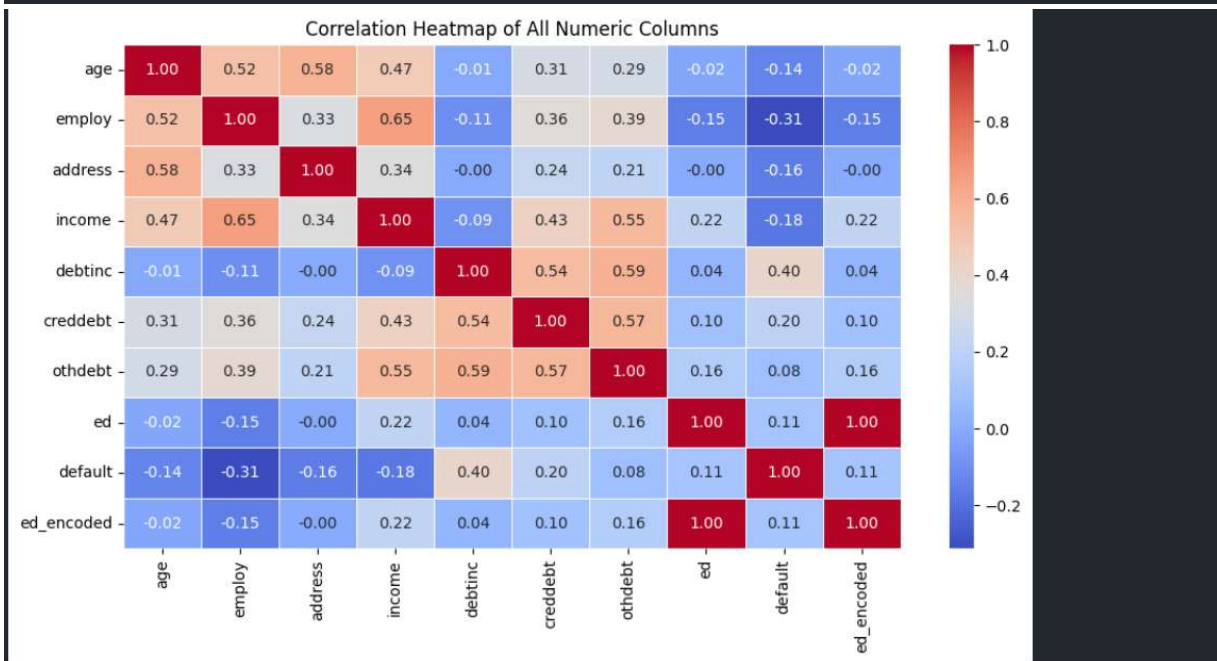
```



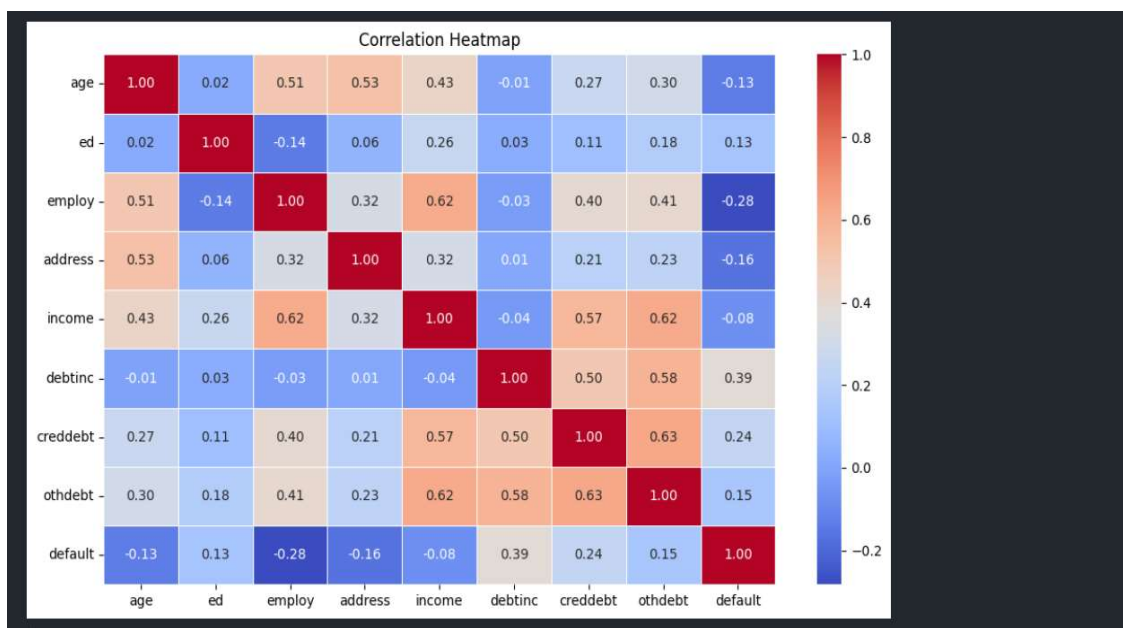
Correlation Matrix:

	age	employ	address	income	debtinc	creddebt	othdebt	ed	\
age	1.00	0.52	0.58	0.47	-0.01	0.31	0.29	-0.02	
employ	0.52	1.00	0.33	0.65	-0.11	0.36	0.39	-0.15	
address	0.58	0.33	1.00	0.34	-0.00	0.24	0.21	-0.00	
income	0.47	0.65	0.34	1.00	-0.09	0.43	0.55	0.22	
debtinc	-0.01	-0.11	-0.00	-0.09	1.00	0.54	0.59	0.04	
creddebt	0.31	0.36	0.24	0.43	0.54	1.00	0.57	0.10	
othdebt	0.29	0.39	0.21	0.55	0.59	0.57	1.00	0.16	
ed	-0.02	-0.15	-0.00	0.22	0.04	0.10	0.16	1.00	
default	-0.14	-0.31	-0.16	-0.18	0.40	0.20	0.08	0.11	
ed_encoded	-0.02	-0.15	-0.00	0.22	0.04	0.10	0.16	1.00	

	default	ed_encoded
age	-0.14	-0.02
employ	-0.31	-0.15
address	-0.16	-0.00
income	-0.18	0.22
debtinc	0.40	0.04
creddebt	0.20	0.10
othdebt	0.08	0.16
ed	0.11	1.00
default	1.00	0.11
ed_encoded	0.11	1.00



3rd csv file:



if there are any missing values in any column of not? How do you want to handle those missing values and why?

In 1st csv file:

Check Missing Values for csv file-1:

```
1 # Check missing values in each column
2 missing_values = df.isnull().sum()
3 print("Missing values per column:\n", missing_values)
4
```

Missing values per column:

age	20
employ	0
address	0
income	37
debtinc	0
creddebt	0
othdebt	0
ed	20
default	0

dtype: int64

+ Code + Markdown

```
[12]: 1 # Fill missing numeric values with median (better for skewed data)
2 df['age'] = df['age'].fillna(df['age'].median())
3 df['income'] = df['income'].fillna(df['income'].median())
4 df['ed'] = df['ed'].fillna(df['ed'].median())
5
```

We use Median because it is not affected by outliers

```
[14]: 1 # Check missing values in each column
2 missing_values = df.isnull().sum()
3 print("Missing values per column:\n", missing_values)
```

Missing values per column:

age	0
employ	0
address	0
income	0
debtinc	0
creddebt	0
othdebt	0
ed	0
default	0

dtype: int64

2nd csv file:

There is no missing value.

Check for Missing Values

+ Code

+ Markdown

[30]:

```
1 print(df2.isnull().sum())  
2
```

```
age          0  
employ       0  
address      0  
income       0  
debtinc      0  
creddebt     0  
othdebt      0  
ed           0  
default      0  
ed_encoded   0  
dtype: int64
```

3rd csv file:

```
[47]: 1 print(df3.isnull().sum())
      2
```

```
age      19
ed       20
employ    0
address   0
income    37
debtinc    0
creddebt   0
othdebt    0
default    2
dtype: int64
```

```
[48]: 1 df3.dropna(inplace=True)
      2
```

```
[55]: 1 df3.fillna({
      2     'ed': df3['ed'].mean(),
      3     'income': df3['income'].mean(),
      4     'default': df3['default'].mean()
      5 }, inplace=True)
      6
      7
```

```
[56]: 1 #now see what happen here
      2 print(df2.isnull().sum())
```

```
age      0
employ    0
address   0
income    0
debtinc    0
creddebt   0
othdebt    0
ed         0
default    0
ed_encoded 0
dtype: int64
```

In conclusion we can say that, the main goal of this analysis was to understand the key factors influencing loan approval decisions. It was highly influenced by applicant income, credit history, loan amount, debt history. Here we could find people income with education level comparison. These findings can help financial institutions optimize their loan approval criteria and improve risk management.

Github Link For Code :

https://github.com/Afarrobo/CSE303_stat_for_DS/blob/main/Dataset%20Analysis/cse303-loan-dataaaa.ipynb